МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

КИЇВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ

БУДІВНИЦТВА І АРХІТЕКТУРИ

КАФЕДРА КІБЕРБЕЗПЕКИ ТА КОМП'ЮТЕРНОЇ ІНЖЕНЕРІЇ

Звіт з лабораторної роботи №2

Дисципліни «Програмування динамічних систем»

Студентки 1-го курсу

групи БІКСм-26

Ненько В. Р.

Перевірив:

Маринський К. В.

Київ-2026

Лабораторна робота №2

Тема: Розробка ноди керування рухом робота через /cmd_vel
Мета: Сформувати практичні навички розробки ноди керування рухом у ROS2, навчитися працювати з повідомленням geometry_msgs/Twist, реалізувати підписку на /cmd_vel, обмеження швидкості та перевірити роботу ноди в симуляції.

Завдання:
1. Підготовка середовища

1.1. Активувати ROS2-середовище.
1.2. Перевірити працездатність команд:

- ros2 node list
- ros2 topic list

1.3. Запустити turtlesim або інший доступний симулятор для тестування.

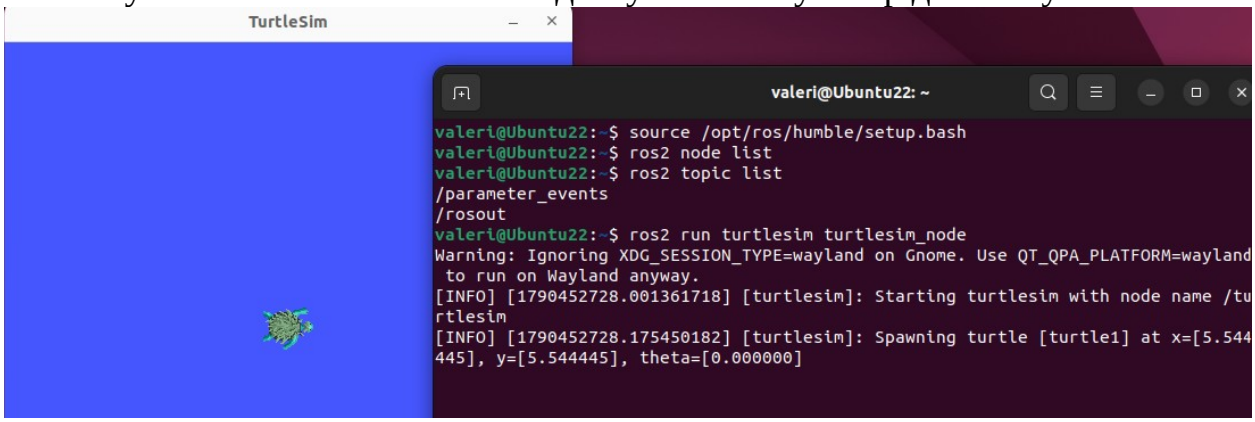


Рис. 1 – Запуск `turtlesim` для тестування

2. Створення пакету Motion Control
2.1. Створити новий пакет типу `ament_python` для реалізації ноди керування рухом.
2.2. Додати залежність від `geometry_msgs`.
2.3. Зібрати workspace та перевірити коректність встановлення пакету.

Спочатку я перейшла в папку `src` свого ROS2 workspace. Потім за допомогою `ros2 pkg create` створила Python-пакет `motion_control` типу `ament_python`. Одразу додала залежності `rclpy` та `geometry_msgs` і створила ноду `motion_node`. Після цього перейшла в корінь workspace, виконала `colcon build` для зборки пакету, підключила результат через `source install/setup.bash` і перевірила командою `ros2 pkg list`, що ROS2 бачить пакет.

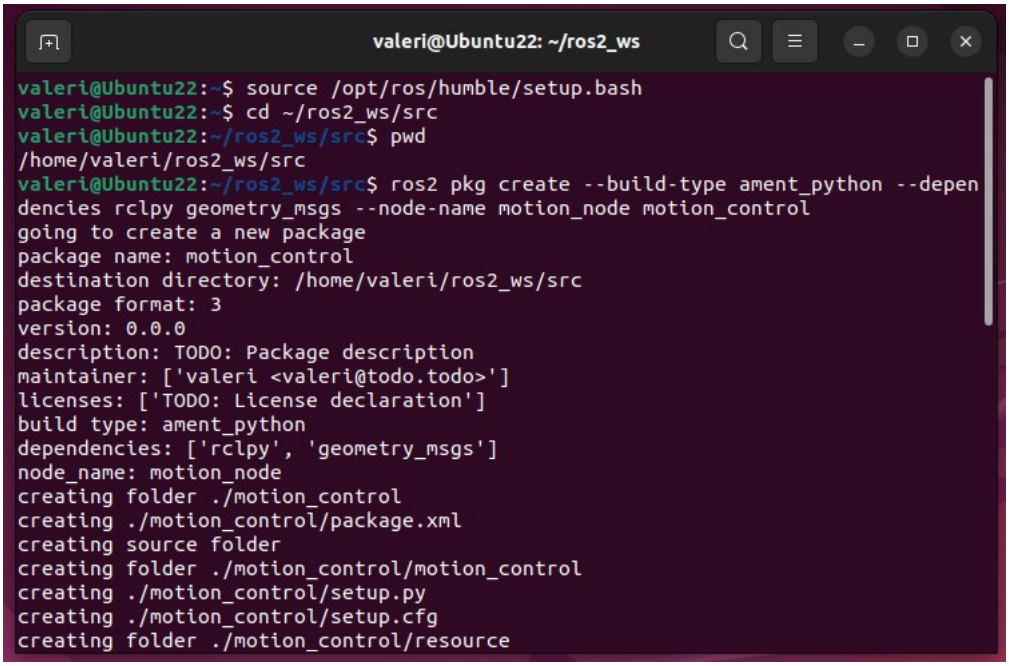


Рис. 2 – Створення пакета `motion_control` типу `ament_python`

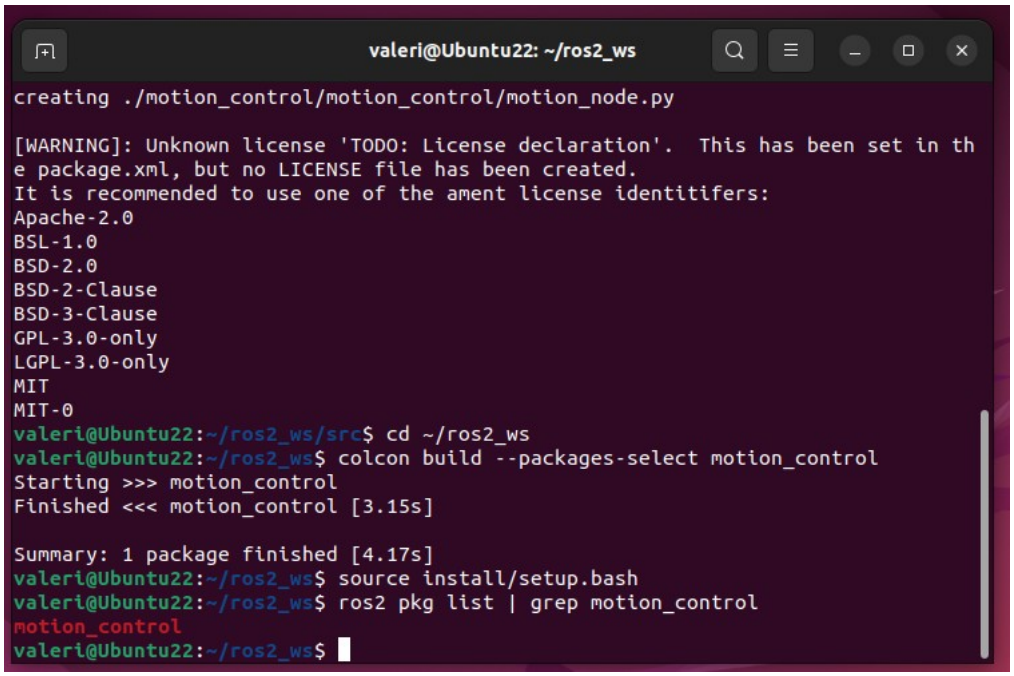


Рис. 3 - Збирання workspace та перевірка встановлення пакета

3. Реалізація Motion Node

3.1. Створити ноду, яка підписується на Topic: /cmd_vel

Для отримання команд руху створено підписку:

```
self.subscription = self.create_subscription(
    Twist,
    '/cmd_vel',
    self.cmd_vel_callback,
    10
)
```

У цьому фрагменті `'/cmd_vel'` — топік, на який підписується нода, `Twist` — тип повідомлення, а `self.cmd_vel_callback` — функція, яка викликається після отримання нового повідомлення. Число `10` задає розмір черги повідомлень.

Для роботи з повідомленням `Twist` використовується імпорт:

```
from geometry_msgs.msg import Twist
```

У callback-функції з отриманого повідомлення беруться необхідні для роботи значення:

```
linear_x = msg.linear.x
angular_z = msg.angular.z
```

`linear.x` відповідає за лінійну швидкість робота, а `angular.z` — за кутову швидкість, тобто швидкість його повороту.

3.2. Використати тип повідомлення: geometry_msgs/msg/Twist

На початку: `from geometry_msgs.msg import Twist`

А далі: `Twist,` тобто наша підписка очікує саме повідомлення типу: `geometry_msgs/msg/Twist`

3.3. Реалізація обмеження швидкості

У ноді встановлено максимальні значення швидкості:

```
self.max_linear_speed = 1.0
self.max_angular_speed = 1.0
```

Під час отримання команди програма перевіряє, чи не перевищують `linear.x` та `angular.z` встановлені межі. Якщо значення перевищує допустимий ліміт, воно обмежується, а в термінал виводиться попередження `WARN`.

Таким чином, нода не просто отримує команди, а додатково перевіряє їх перед подальшою обробкою.



Рис. 4 - Перехід до директорії пакета та відкриття файлу `motion_node.py` для редагування

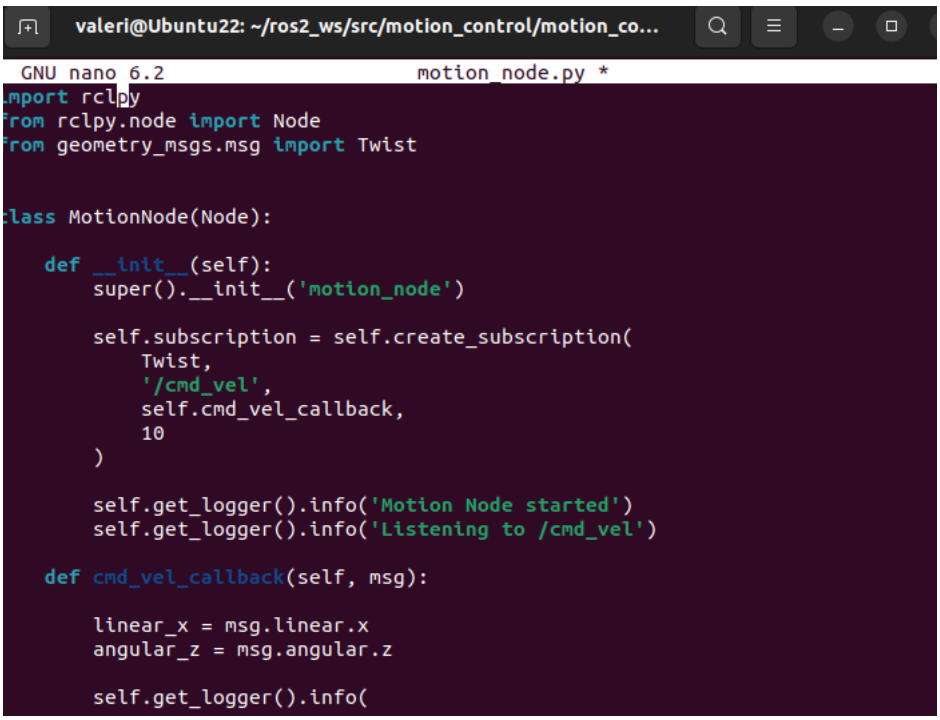


Рис. 5 – Фрагмент коду ініціалізації `MotionNode` та створення підписки на топік `/cmd_vel`

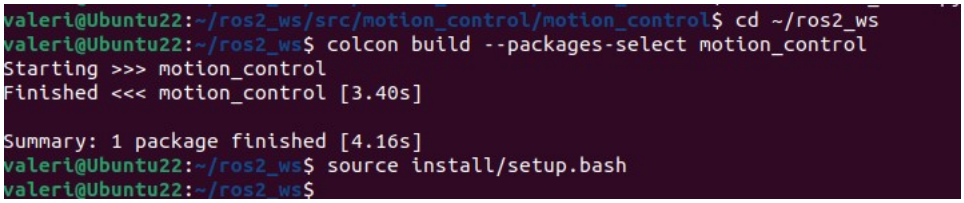


Рис. 6 - Повторне збирання пакета `motion_control` після змін у коді

3.4. Логування та перевірка роботи ноди через логер ROS2 (INFO-рівень).

Для виведення інформації використовується ROS2 logger:

```
self.get_logger().info(
    f'Received: linear.x={linear_x:.2f}, '
    f'angular.z={angular_z:.2f}'
)
```

`self.get_logger().info()` — це саме INFO-рівень ROS2 logger.

Наприклад, коли ми надішлемо:

```
linear.x = 0.5
angular.z = 0.2
```

у терміналі ноди отримаємо:

```
[INFO] [motion_node]: Received: linear.x=0.50, angular.z=0.20
```

```
valeri@Ubuntu22:~/ros2_ws$ ros2 run motion_control motion_node
[INFO] [1790617952.360043693] [motion_node]: Motion Node started
[INFO] [1790617952.360789625] [motion_node]: Listening to /cmd_vel
```

Рис. 7 - Демонстрація успішного запуску `Motion Node` та підписки на топік `/cmd_vel`.

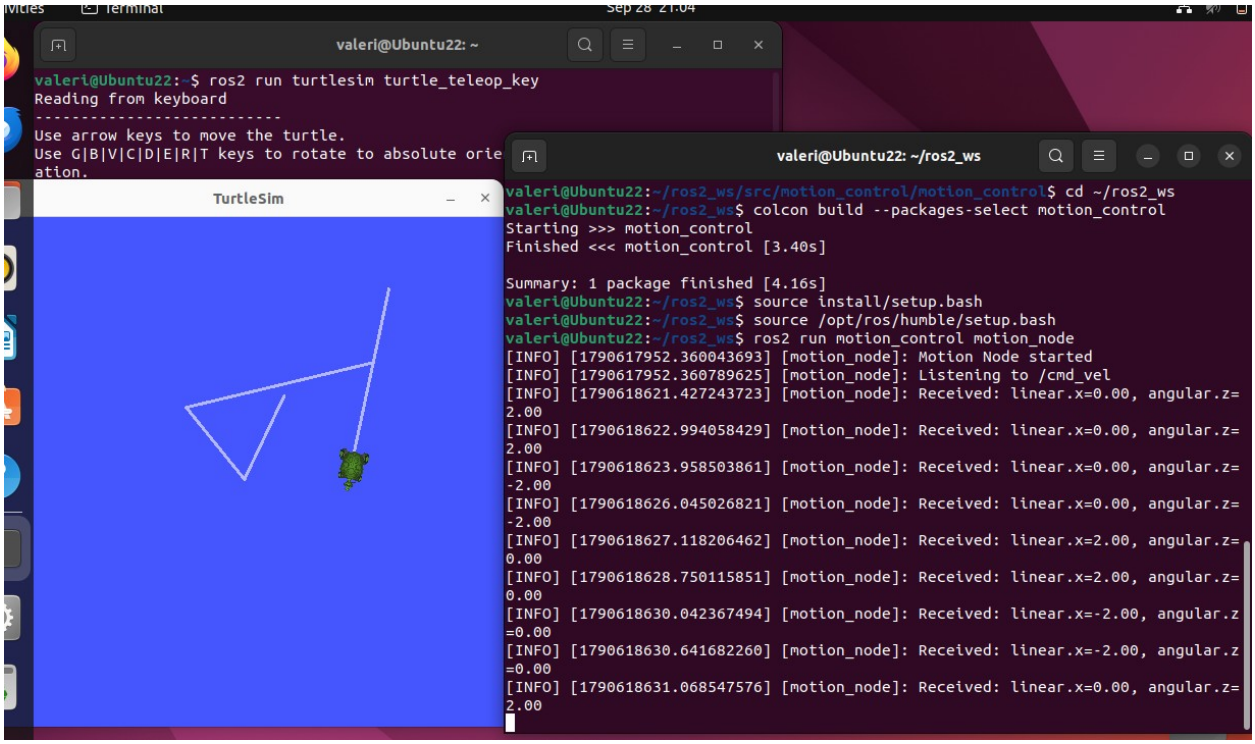


Рис. 8 - Вивід отриманих значень лінійної та кутової швидкості через логер ROS2 (INFO-рівень).

Стандартний `turtle_teleop_key` публікує команди керування в `turtle1/cmd_vel`, а моя нода `motion_node` підписана на `/cmd_vel`. Тому я використала `--remap`, щоб перенаправити `turtle1/cmd_vel` у `/cmd_vel`. Після цього команди з клавіатури почали надходити до моєї ноди, де вони оброблялися та перевірялися на перевищення швидкості.

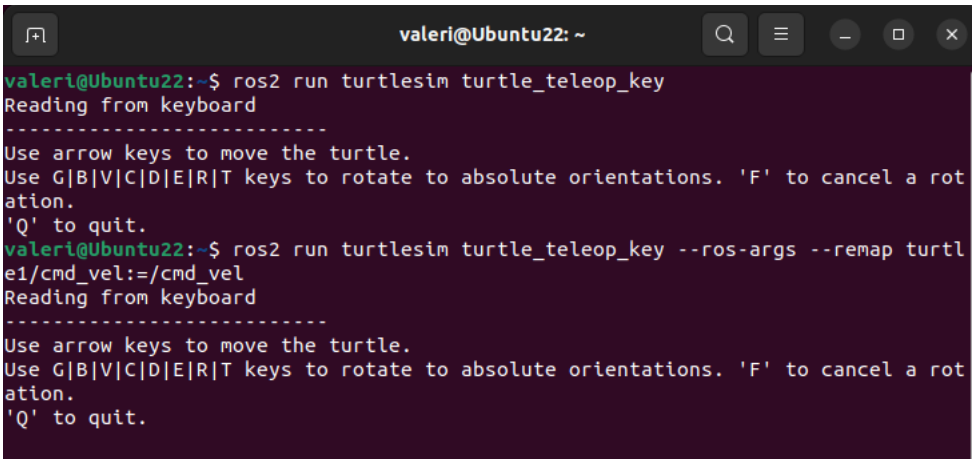


Рис. 9 - Запуск вузла керування з клавіатури із перенаправленням (remap) у топік `/cmd_vel`.

4. Реалізація обмеження швидкості

4.1. Додати перевірку допустимих меж швидкості.

4.2. Якщо значення перевищує встановлений поріг — обмежити його.

4.3. У разі перевищення виводити WARN-повідомлення.

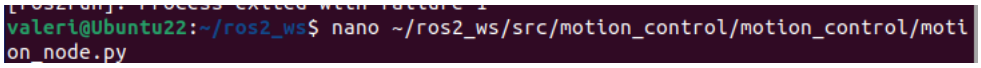


Рис. 10 - Відкриття файлу `motion_node.py` для додавання обмеження швидкості



Рис. 11 – Додавання параметрів максимальної лінійної та кутової швидкості у конструктор класу

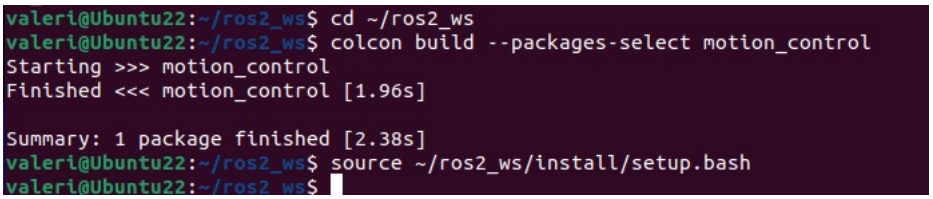


Рис. 12 - Збирання workspace та оновлення середовища виконання

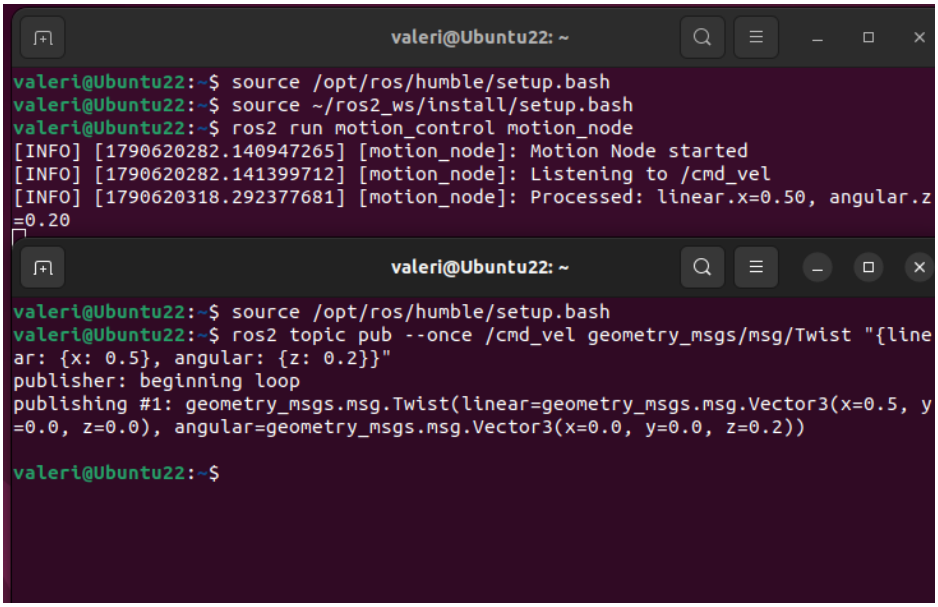


Рис. 13 - Тестування ноди шляхом публікації тестового повідомлення у топік `/cmd_vel`

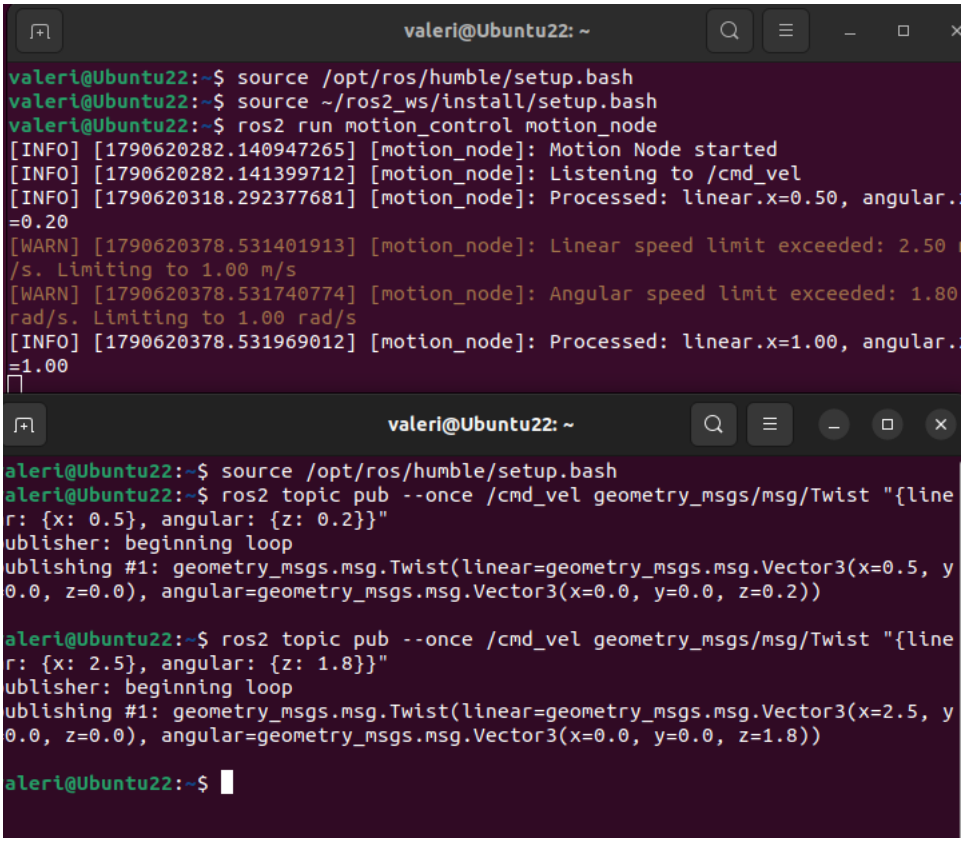


Рис. 14 - Перевірка роботи обмеження швидкості та виведення попереджень (WARN-повідомлень) у разі перевищення лімітів

5. Тестування ноди

5.1. Запустити Motion Node.

5.2. Подати команди у /cmd_vel через teleop або через ros2 topic pub.

5.3. Перевірити, що нода отримує повідомлення та коректно їх обробляє.

5.4. Переконатися, що обмеження швидкості працює.

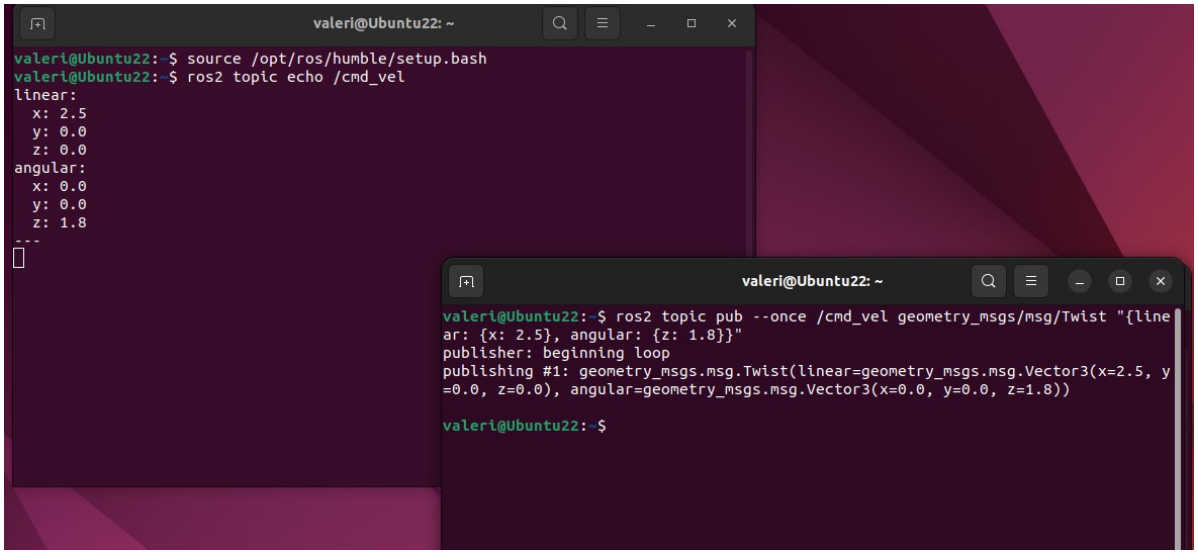


Рис. 15 - Перевірка переданих даних за допомогою команди `ros2 topic echo /cmd_vel`

Висновки:

Сформувала практичні навички розробки ноди керування рухом у ROS2, навчилася працювати з повідомленням `geometry_msgs/Twist`, реалізувала підписку на `/cmd_vel`, обмеження швидкості та перевірила роботу ноди в симуляції.

Під час лабораторної роботи я створила ноду motion_node, яка отримує команди руху через топік `/cmd_vel`. Нода працює з повідомленнями `geometry_msgs/Twist` та отримує значення лінійної і кутової швидкості.

Також я додала обмеження швидкості. Якщо отримане значення перевищує встановлений ліміт, програма виводить попередження WARN і обмежує швидкість. Роботу ноди я перевірила за допомогою `teleop` та команди `ros2 topic pub`, а отримані повідомлення перевірила через `ros2 topic echo`.

Різниця між ручним і програмним керуванням: при ручному керуванні команди задає користувач, наприклад за допомогою клавіатури, а при програмному їх формує та обробляє сама програма. Такий підхід у подальшому дозволяє автоматизувати рух робота та додавати до нього різні алгоритми керування.